

데이터베이스시스템

Proj #1 보고서

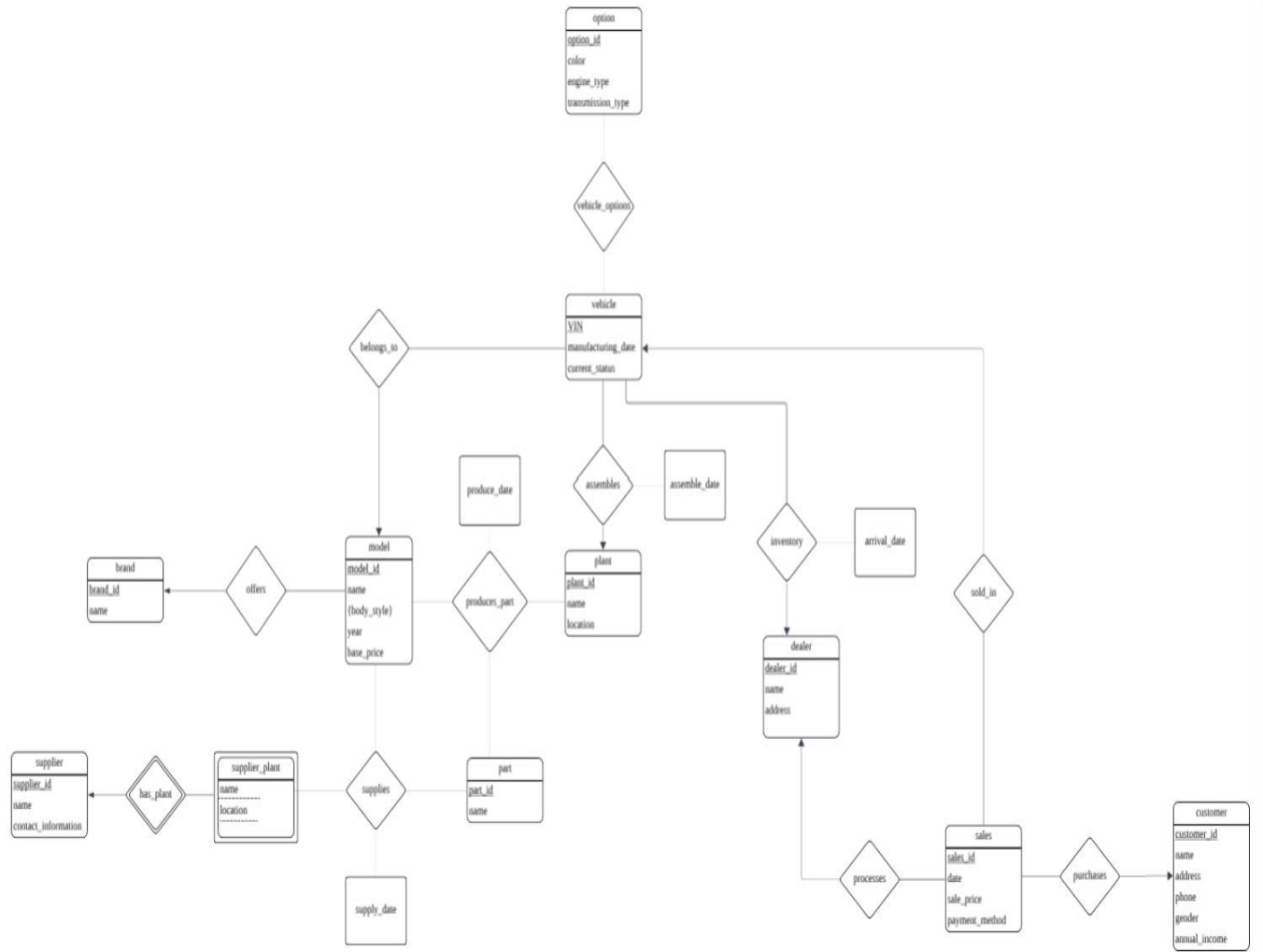
정성원 교수님

20191274

수학과

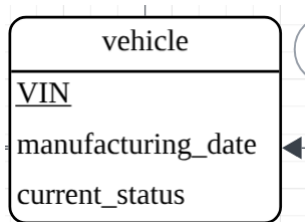
장유빈

0. E-R diagram



1. Entity and Relationship Justification and Mapping Cardinality Explanation

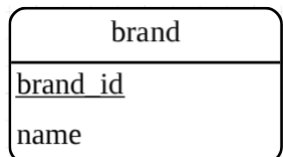
Vehicle 엔티티는 다음과 같다.



PK로 VIN(Vehicle Identification Number)를 갖고,
속성으로 제조일자를 나타내는 manufacturing_date와
현재 상태를 나타내는 current_status가 있다.

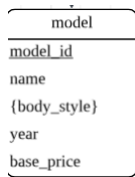
Relational Scheme에서는 FK를 나타낼 assembly_plant는 E-R diagram에서는 나타내지 않았다.

Brand 엔티티는 다음과 같다.



PK로 brand_id를 갖고,
추가적인 속성으로 브랜드명을 나타내는 name 속성을 만들었다.

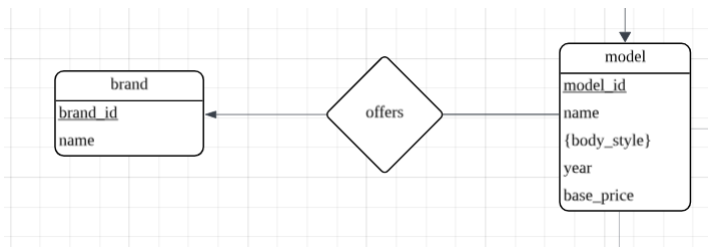
Model 엔티티는 다음과 같다.



PK로 model_id를 갖는다.

추가적인 속성으로는 name, {body_style}, year, base_price를 갖는다.

각 model은 여러가지 body_style을 가질 수 있기 때문에, multivalued 속성으로 설정하였다.



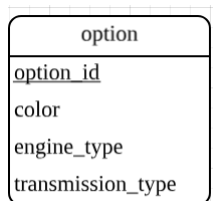
또한, 각 brand마다 여러가지 model을 제공하기 때문에, offers 라는 relationship set을 만들었다.

Mapping Cardinality는 1 : N이 되고,

모든 model은 반드시 어떤 brand에 속해야 하기 때문에,

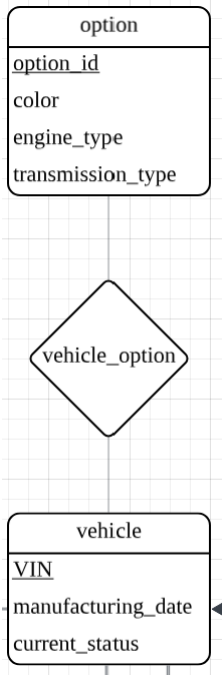
Total Participation으로 설정하였다.

Option 엔티티는 다음과 같다.



PK로 option_id를 갖는다.

추가적인 속성으로는 color, engine_type, transmission_type을 갖는다.



조건에서 Vehicles may have various options 이라고 하였다.

따라서, 한 vehicle은 여러 option을 가질 수 있고,

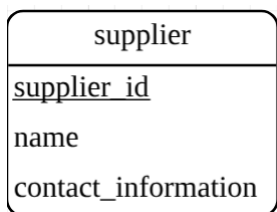
동시에, 한 option은 여러 vehicle에 나타날 수 있다.

따라서, N : M의 Mapping Cardinality를 가지도록 하였다.

즉, 여기서 나는 option을 추가적인 속성으로 두지 않고,

또 다른 엔티티로 설정한 것이다.

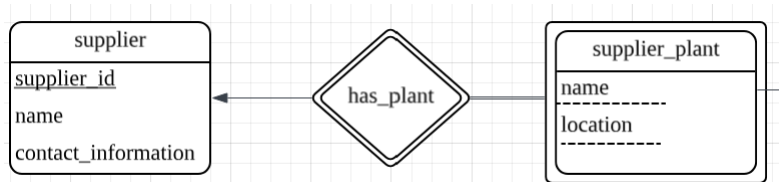
외부 공급자를 의미하는 supplier 엔티티는 다음과 같다.



PK로 supplier_id를 가지고,

추가적인 속성으로 name, contact_information을 가진다.

외부 공급자는 여러 공장을 가질 수 있다고 하였다.



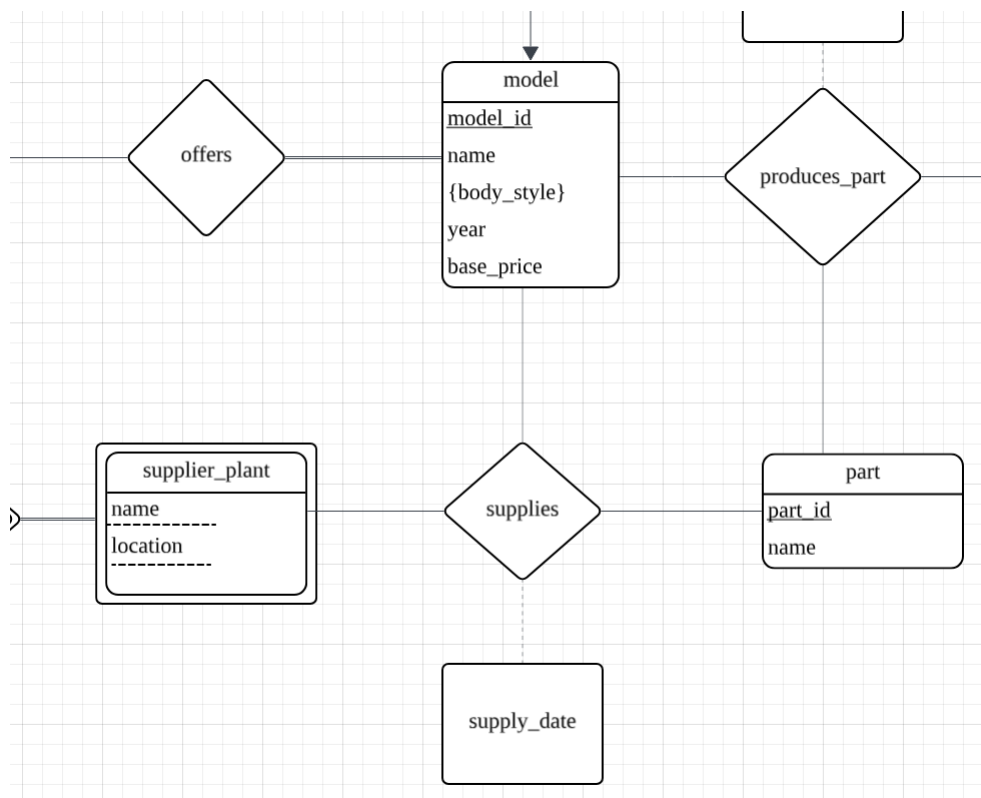
이러한 공장은 supplier에 의존적이다.

Supplier가 소유하고 있다고 생각하면 될 것이다.

따라서, supplier_plant라는 weak entity set을 만들었고, has_plant 라는 identifying relationship을 만들었다.

또한, supplier는 여러 공장을 소유하고 있기 때문에, 1 : N의 Mapping Cardinality를 가지고 있다고 보면 된다.

Supplier_plant 쪽은 Total Participation으로 설정하였다.



또한, 조건에서 External suppliers supply certain parts for certain models 이라고 하였다.

따라서, 부품을 의미하는 part 엔티티를 따로 만들었다.

PK는 part_id를 가지며, 추가적인 속성으로 부품명을 의미하는 name 속성을 만들었다.

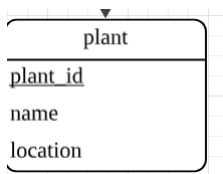
또한, External Supplier가 소유하고 있는 supplier_plant에서 해당 part를 해당 model에 제공하기 때문에, Ternary Relationship인 suppliers 을 만들었다.

여기서, model에 화살표를 고민하였지만,

여러 model에 부품을 제공할 수 있다고 생각하기 때문에 화살표를 따로 작성하지 않았다.

Suppliers라는 relationship에서 부품 납품일자를 나타내는 supply_date 속성을 추가로 만들었다.

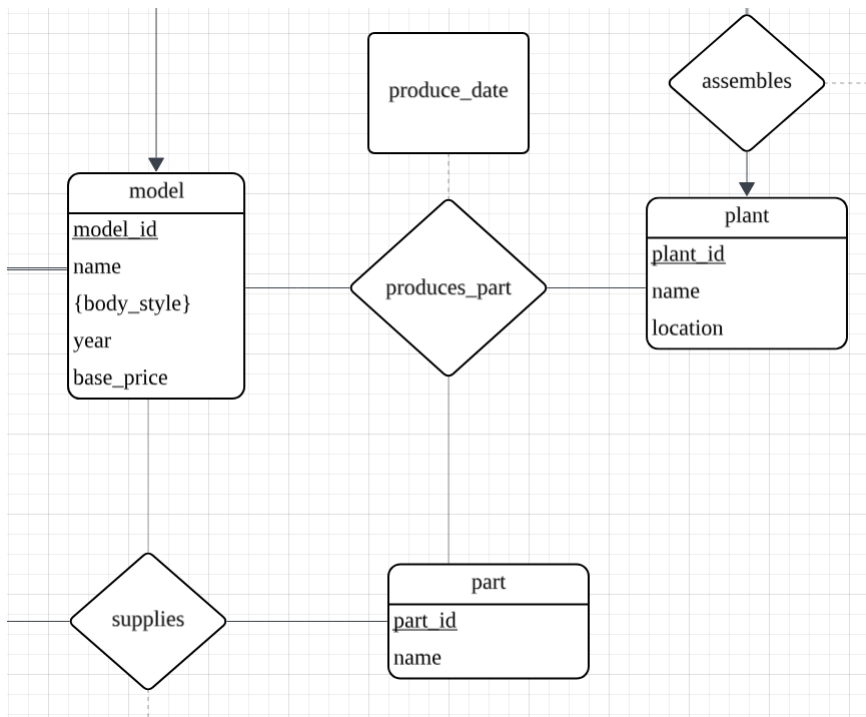
이제, 외부 공급자가 소유하고 있는 공장이 아닌 회사가 소유하고 있는 공장을 의미하는 plant 엔티티를 살펴보자.



PK로 plant_id를 가지며,

추가적인 속성으로는 공장 이름을 나타내는 name 속성과 공장 위치를 나타내는 location 속성을 가지고 있도록 하였다.

마찬가지로 해당 공장에서도 부품을 해당 모델에 제공할 수 있다고 조건에 나와있다.



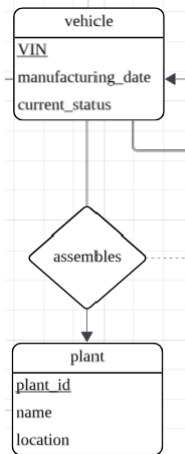
따라서, plant, part, model 사이에 또 다른 Ternary Relationship인 produces_part를 만들었다.

아까와 마찬가지로, 해당 plant에서 해당 part를 model에 제공 및 공급한다고 이해하면

될 것 같다.

Produces_part의 추가적인 속성으로 produce_date를 만듦으로써, 해당 plant가 해당 part를 model에 언제 제공하였는지 나타낼 수 있도록 하였다.

또한, 몇몇 공장은 vehicle을 assemble(조립)한다고 하였으므로,



Assembles라는 relationship을 만들었다.

Vehicle쪽을 Total Participation으로 설정하였다.

모든 vehicle은 해당 plant에서 조립된다는 의미이다.

Mapping Cardinality는 1 : N이다.

즉, 한 plant에서 여러 vehicle을 조립한다는 의미이다.

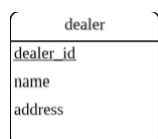
이때, assembles 관계에 추가적인 속성인 assemble_date를 설정함으로써,

조립 날짜가 언제인지 확인할 수 있도록 하였다.

참고로, plant 엔티티는 PK로 plant_id를 가지며,

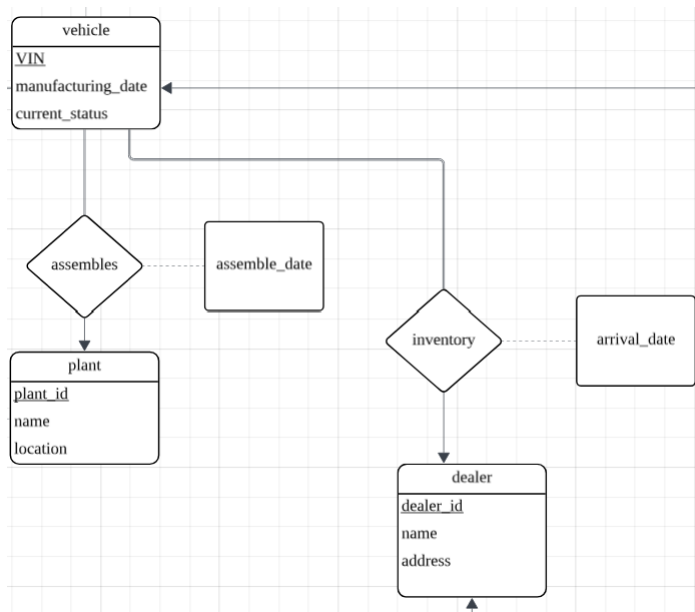
추가적인 속성으로 공장 이름을 의미하는 name과 공장 위치를 의미하는 location 속성을 가지도록 하였다.

이제, dealer 엔티티를 살펴보도록 하자.



PK로 dealer_id를 가지며,

추가적인 속성으로 딜러의 이름을 의미하는 name과 address를 가지도록 하였다.



Dealer의 속성으로 inventory_of_vehicles, 즉, 차량의 재고를 넣는 대신

Inventory라는 관계로 표현하였다.

즉, dealer가 vehicle들을 재고로 관리하고 있다는 의미입니다.

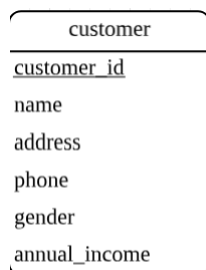
Inventory 관계에서 추가적인 속성으로 arrival_date를 설정함으로써,

해당 dealer에게 해당 차량이 입고된 날짜가 언제인지 확인할 수 있게 하였다.

즉, 해당 차량을 관리하기 시작한 날짜가 언제인지 확인할 수 있도록 하였다.

이 inventory 관계를 통해 해당 dealer가 관리하고 있는 차량의 수 또한 알아낼 수 있을 것이다.

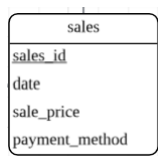
이제 customer 엔티티를 한번 살펴보자.



PK로 customer_id를 가지며,

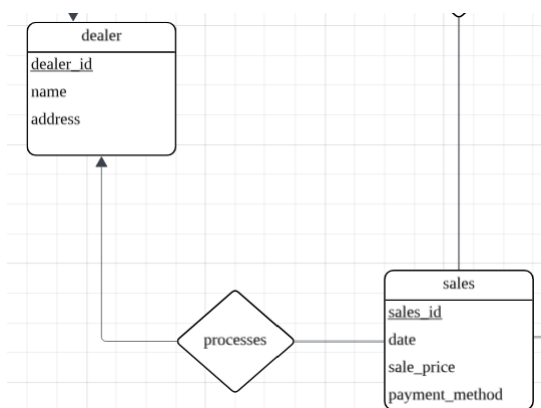
추가적인 속성으로는 name, address, phone, gender, annual_income을 가지도록 하였다.

Sales 엔티티를 한번 살펴보자.

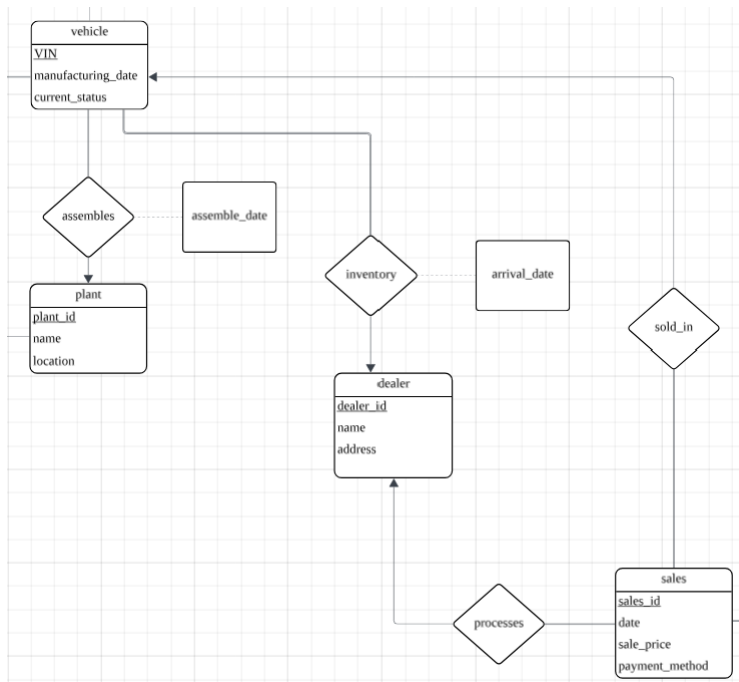


PK로는 sales_id를 가진다.

또한, 추가적인 속성으로 거래 일자를 의미하는 date, sale_price, payment_method를 가진다.

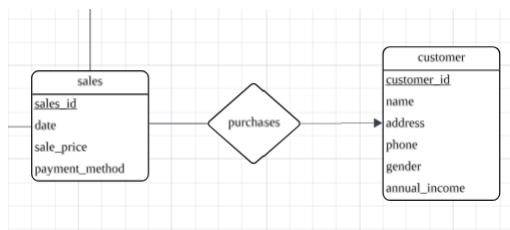


해당 dealer가 여러 sales를 진행할 수 있기 때문에, Mapping Cardinality는 1 : N으로 설정하였다. 또한, sales쪽은 Total Participation으로 설정하였다.



Sold_in 관계를 한번 살펴보자.

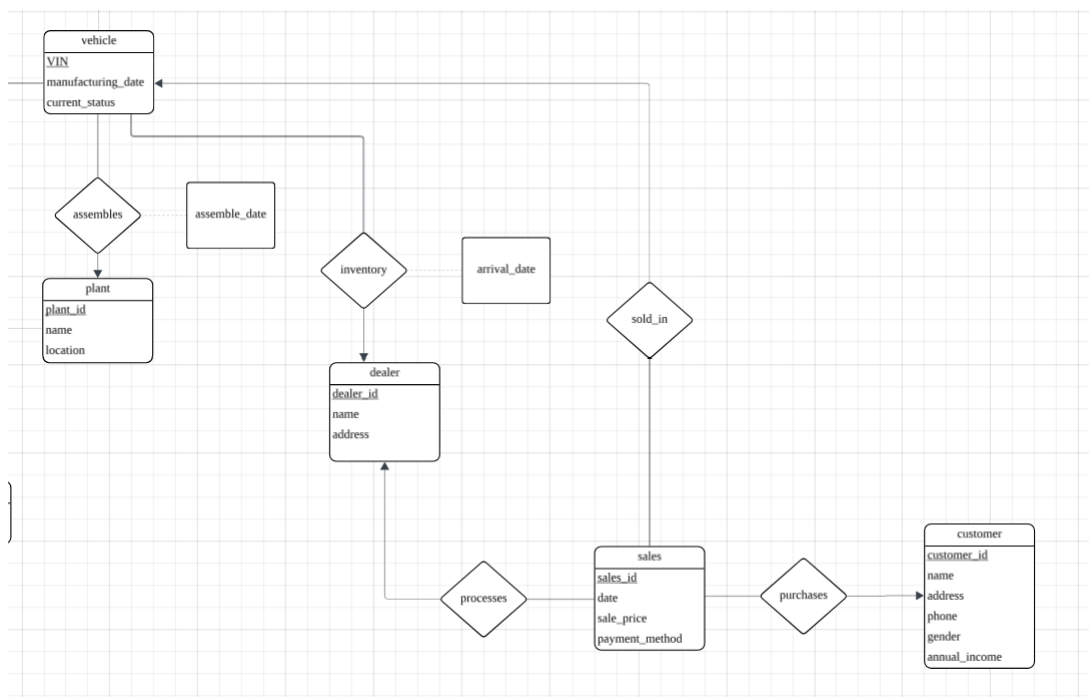
해당 vehicle이 어떤 sales에서 거래되었는지 알 수 있을 것이다.



Purchases 관계를 한번 살펴보자.

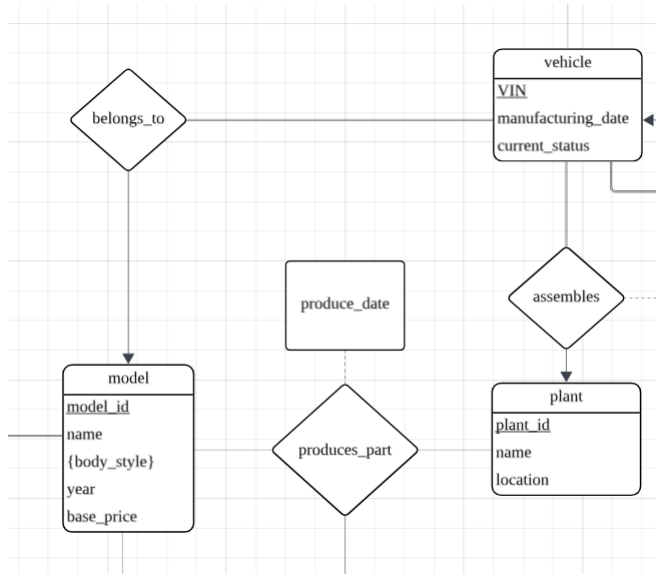
해당 customer가 여러 번 차량을 구매할 수 있기 때문에,

Mapping Cardinality는 customer 기준으로 1 : N의 관계로 설정하였다.



전체적으로 보면 Ternary Relationship으로 설정하는 것도 괜찮을 수 있지만,

Sales를 엔티티로 설정하였기 때문에, 3개의 Binary Relationship으로 표현했다.



모든 vehicle은 특정 model에 속해야 한다.

따라서, belongs_to 관계를 따로 만들었다.

1 : N의 Mapping Cardinality를 가진다.

또한, 모든 vehicle은 반드시 특정 model에 속해야 한다.

따라서, Total Participation으로 설정하였다.

참고로, Relational Schema로 변환할 때에는,

Vehicle이 model_id라는 FK를 가져야 할 것이다.

2. Query Support Description

1. **Sales Trends:** *"Show sales trends for various brands over the past 3 years, by year, month, and week. Then break these data out by gender of the buyer and then by income range."*

가장 먼저 분석의 중심이 되는 sales 테이블을 가져오고, 필요한 속성을 가진 주변 테이블들을 결합합니다.

FROM

sales s

-- 중심이 되는 판매 기록 테이블을 가져옵니다.

JOIN

vehicle v ON s.VIN = v.VIN

JOIN

model m ON v.model_id = m.model_id

JOIN

brand b ON m.brand_id = b.brand_id

-- "various brands"를 알기 위해, ERD의 다리(vehicle ~> model ~> brand)를 건너 결합합니다.

JOIN

customer c ON s.customer_id = c.customer_id

-- "gender of the buyer"와 "income range"를 알기 위해 고객 테이블을 결합합니다.

결합한 데이터 셋 중에서, 분석 조건에 맞지 않는 불필요한 데이터는 쳐냅니다.

WHERE

s.date >= CURRENT_DATE - INTERVAL '3 YEARS'

-- "over the past 3 years" 요구사항에 맞춰, 최근 3년 이내의 판매 기록만 필터링합니다.

문제에서 제시한 기준("by ~")에 따라 파티션을 나눕니다.

이때 날짜 추출과 소득 구간화 작업이 함께 들어갑니다.

GROUP BY

b.brand_id, -- by various brands.

EXTRACT(YEAR FROM s.date), -- by year.

EXTRACT(MONTH FROM s.date), -- by month.

EXTRACT(WEEK FROM s.date), -- by week.

C.gender, -- break out by gender.

CASE

```
WHEN c.annual_income < 30000 THEN 'Low'
```

```
WHEN c.annual_income BETWEEN 30000 AND 60000 THEN 'Middle'
```

```
ELSE 'High'
```

```
END -- break out by income range (연속형 데이터를 범주형으로 변환).
```

이제, 그룹화된 기준 속성들을 출력하고, 각 그룹별로 "Sales Trend(판매 동향)"를 나타낼 집계 함수를 적용합니다.

```
SELECT
```

```
    B.brand_id,
```

```
    EXTRACT(YEAR FROM s.date) AS sales_year,
```

```
    EXTRACT(MONTH FROM s.date) AS sales_month,
```

```
    EXTRACT(WEEK FROM s.date) AS sales_week,
```

```
    C.gender,
```

```
    CASE
```

```
        WHEN c.annual_income < 30000 THEN 'Low'
```

```
        WHEN c.annual_income BETWEEN 30000 AND 60000 THEN 'Middle'
```

```
        ELSE 'High'
```

```
    END AS income_range,
```

```
    COUNT(s.sales_id) AS total_sales_volume -- 트렌드를 보여주기 위한 총 판매 건수 (집계).
```

```
    SUM(s.sale_price) AS total_sales_revenue -- 트렌드를 보여주기 위한 총 매출액 (집계).
```

이제, 마지막으로 결과를 보기 좋게 정렬합니다.

```
ORDER BY
```

B.brand_id, sales_year, sales_month, sales_week

이렇게 접근한다면, 해당 Query를 실행할 수 있을 것입니다.

2. **Defective Part Tracking:** *"Suppose that it is found that transmissions made by supplier Getrag between two given dates are defective. Find the VIN of each car containing such a transmission and the customer to which it was sold. If your design allows, suppose the defective transmissions all come from only one of Getrag's plants."*

어떻게 접근하면 이 Query를 실행할 수 있을 지 대략적인 Query를 한번 작성해보겠습니다.

SELECT

V.VIN,

C.name AS customer_name,

C.phone AS customer_phone

FROM

Supplier s

JOIN

Supplier_plant spl on s.supplier_id = spl.supplier_id

-- 약한 엔티티와 식별 관계(has_plant) 조인.

JOIN

Supplies sp on spl.supplier_id = sp.supplier_id AND spl.name = sp.plant_name

-- 수정된 ERD 반영: suppliers가 supplier_plant와 직접 연결됨 (복합키 조인 가정).

JOIN

Part p on sp.part_id = p.part_id

JOIN

Model m on sp.model_id = m.model_id

JOIN

Vehicle v on m.model_id = v.model_id

JOIN

Sales sl on v.VID = sl.VIN

JOIN

Customer c on sl.customer_id = c.customer_id

WHERE

S.name = 'Getrag' -- 1. 공급사가 Getrag.

AND p.name = 'transmission' -- 2. 부품이 변속기.

AND sp.supply_date BETWEEN '시작일자' AND '종료일자' -- 3. 특정 기간.

AND spl.name = '결함이_발견된_특정_공장_이름'; -- 4. 특정 공장 지목.

즉, 중심이 되는 supplier 테이블에서 시작합니다.

차례대로 supplier_plant, supplies, part, model을 JOIN합니다.

그리고, vehicle, sales, customer를 JOIN합니다.

이렇게 된다면, WHERE 조건을 통해서, 해당 결함이 있는 변속기가 장착된 vehicle을 구매한 customer 정보까지 얻을 수 있을 것입니다.

SELECT 절에서는 v.VIN과 customer의 name, customer의 phone 정보를 나열하도록 하였습니다.

즉, E-R diagram을 통해 이런 식으로 쿼리를 가능하게 할 수 있습니다.

3. **Top Brands by Revenue:** *"Find the top 2 brands by dollar-amount sold in the past year."*

SELECT

B.name as brand_name,

SUM(s.sale_price) as total_revenue

FROM

Sales s

JOIN

Vehicle v on s.VIN = v.VIN

JOIN

Model m on v.model_id = m.model_id

JOIN

Brand b on m.brand_id = b.brand_id

WHERE

S.date >= CURRENT_DATE - INTERVAL '1 YEAR' -- 최근 1년 데이터 필터링.

GROUP BY

B.brand_id, b.name -- 브랜드별로 그룹화 진행.

ORDER BY

Total_revenue desc -- 매출액 기준 내림차순 진행.

LIMIT 2; -- 상위 2개 브랜드만 추출.

즉, 기준이 되는 sales 테이블에서 시작하여,

브랜드 정보를 알기 위해,

Vehicle, model, brand 테이블을 순차적으로 JOIN을 진행합니다.

그리고, WHERE 조건절을 통하여, 지난 1년간의 판매 기록만 필터링합니다.

그리고, GROUP BY를 통하여, 브랜드별로 그룹화를 진행합니다.

그리고, 브랜드별 매출액을 기준으로 내림차순을 진행합니다.

그리고, LIMIT 2 를 통하여 상위 2개 브랜드만 추출을 진행합니다.

4. **Top Brands by Unit Sales:** *"Find the top 2 brands by unit sales in the past year."*

SELECT

B.name as brand_name,

COUNT(s.sales_id) as total_unit_sales -- 금액을 더하는 SUM 대신에, 판매 대수 및
건수를 세는 COUNT 사용합시다.

FROM

Sales s

JOIN

Vehicle v on s.VIN = v.VIN

JOIN

Model m on v.model_id = m.model_id

JOIN

```
Brand b on m.brand_id = b.brand_id

WHERE

S.date >= CURRENT_DATE - INTERVAL '1 YEAR' -- 최근 1년 데이터 필터링.

GROUP BY

B.brand_id, b.name -- 브랜드별로 묶기를 진행합니다.

ORDER BY

Total_unit_sales DESC -- 판매 대수 기준 내림차순 진행.

LIMIT 2; -- 상위 2개 브랜드만 추출.
```

3번과 매우 유사하게 접근할 수 있습니다.

브랜드별로 그룹화하는 것도 똑같습니다.

하지만, 총 매출액을 구하는 SUM 대신에,

브랜드별 총 판매 대수를 구하려면, COUNT 집계 함수를 이용해야 한다는 점이 다릅니다.

```
SELECT

    EXTRACT(MONTH FROM s.date) AS best_selling_month,

    COUNT(s.sales_id) AS total_unit_sales

FROM

    Sales s

JOIN

    Vehicle v on s.VIN = v.VIN

JOIN

    Model m on v.model_id = m.model_id

WHERE

    M.body_style = 'convertible' -- 컨버터블 차량만 필터링을 진행합니다.

GROUP BY

    EXTRACT(MONTH FROM s.date) -- 월별로 그룹화를 진행해야 합니다.

ORDER BY

    Total_unit_sales DESC -- 판매 대수 내림차순 진행.

LIMIT 1; -- 가장 많이 팔린 1개의 달만 추출을 진행합니다.
```

중심이 되는 sales 테이블에서 시작하여,

Body_style 정보를 얻기 위해,

차례대로, vehicle, model 테이블을 JOIN을 진행합니다.

그리고, WHERE 조건절을 통하여, m.body_style = 'convertible'을 통하여,

컨버터블 차량만 필터링을 진행합니다.

그리고, 월별로 그룹화를 진행합니다.

월별 총 판매 대수를 기준으로 내림차순을 진행하고,

LIMIT 1 으로 가장 많이 팔린 월만 추출해냅니다.

6. **Dealer Inventory Efficiency:** *"Find those dealers who keep a vehicle in inventory for the longest average time."*

SELECT

D.dealer_id,

D.name,

AVG(s.date - v.arrival_date) AS avg_inventory_time - vehicle 테이블에서 직접 arrival_date를 가져옴.

FROM

Dealer d

JOIN

Vehicle v on d.dealer_id = v.dealer_id -- 별도의 inventory 테이블 없이 1 : N 직접
조인.

JOIN

Sales s on v.VIN = s.VIN

GROUP BY

D.dealer_id,

D.name

ORDER BY

Avg_inventory_time DESC

LIMIT 1;

즉, 중심이 되는 dealer 테이블에서 시작합니다.

그리고, vehicle 테이블과 JOIN을 진행하고,

그리고, sales 테이블과 JOIN을 진행합니다.

그리고, d.dealer_id, d.name을 기준으로 즉, 딜러별로 그룹화를 진행합니다.

그리고, 각 딜러별로 평균 보관 기간을 구해냅니다.

평균 보관 기간을 내림차순으로 정렬한 후에,
가장 오랜 보관 기간을 가지고 있는 딜러들을 추출해냅니다.

7. **Supplier Coverage:** *"Which supplier provides parts for the most number of distinct models across all brands?"*

SELECT

S.supplier_id,

S.name,

COUNT(DISTINCT sp.model_id) AS distinct_model_count

-- 한 공급자가 한 모델에 여러 부품을 제공할 수 있기 때문에,

```

-- 중복을 제거해야 합니다.

FROM

    Supplier s

JOIN

    Supplier_plant spl on s.supplier_id = spl.supplier_id

JOIN

    Supplies sp on spl.supplier_id = sp.supplier_id AND spl.name = sp.plant_name

-- 참고로, supplies 테이블 안에 이미 model_id가 왜래키로 존재하기 때문에,
-- 단순히 카운트가 목적이라면, part나 model 테이블까지 굳이 JOIN하지 않아도
된다고 합니다.

GROUP BY

    S.supplier_id,

    S.name

ORDER BY

    Distinct_model_count DESC

LIMIT 1;

```

먼저, 중심이 되는 supplier 테이블에서 시작합니다.

차례대로 supplier_plant, supplies 테이블을 JOIN을 진행합니다.

(필요에 따라, part, model을 조인하여 흐름을 명확히 할 수도 있습니다.).

그리고, s.supplier_id, s.name을 기준으로 즉, 공급자별로 그룹화를 진행합니다.

이때, 한 공급자가 같은 모델에 여러 부품을 납품할 수 있습니다.

COUNT(DISTINCT model_id)를 사용하여 중복을 제거하고,

각 공급자별로 얼마나 많은 고유한 모델에 부품을 제공했는지 셉니다.

이것을 기준으로 내림차순을 진행하고,

LIMIT 1 처리로 가장 많은 종류의 모델에 부품을 제공한 상위 공급자 1명을 찾아냅니다.

이렇게 접근하면 해당 Sample Query를 처리할 수 있을 것입니다.